Experiment - 8

Student Name: Manjot Singh
UID: 23BCS12549
Branch: BE-CSE
Section/Group: KRG_2B
Semester: 6th
Date of Performance: 25/3/26
Subject Name: System Design
Subject Code: 23CSH-314

Aim:

To design and analyze a scalable food delivery system that enables users to browse restaurants, place orders, track deliveries in real-time, and handle payments efficiently while ensuring high availability, scalability, and low latency.

Objectives:

1) To understand system design principles for large-scale food delivery platforms.
2) To design APIs for user, restaurant, and order management.
3) To identify functional and non-functional requirements.
4) To apply distributed system concepts like CAP theorem.
5) To design a scalable and fault-tolerant architecture.

Procedure-

1) Identify functional requirements of a food delivery system.
2) Define non-functional requirements such as scalability, availability, and latency.
3) Analyze CAP theorem trade-offs.
4) Identify core entities such as users, restaurants, orders, and delivery agents.
5) Design system architecture using Draw.io.
6) Validate system behavior for real-time order tracking and delivery.

Functional Requirements -

User Management
- User should be able to register and login.
- User should be able to update profile and address.

Restaurant Management
- System should display nearby restaurants.
- Restaurant should manage menu items and pricing.
- System should allow search and filtering (rating, cuisine, price).

Order Management
- User should be able to place an order.
- User should be able to cancel an order.
- System should maintain order history.
- System should show order status (placed, preparing, out for delivery, delivered).

Delivery Management
- System should assign delivery agents based on proximity.
- Delivery agent should be able to accept/reject orders.
- Real-time location tracking of delivery agent.

Payment System
- Support multiple payment modes (UPI, cards, wallet, COD).
- Ensure secure transactions.

Rating & Review
- User should be able to rate restaurants and delivery agents.

Non-functional Requirements

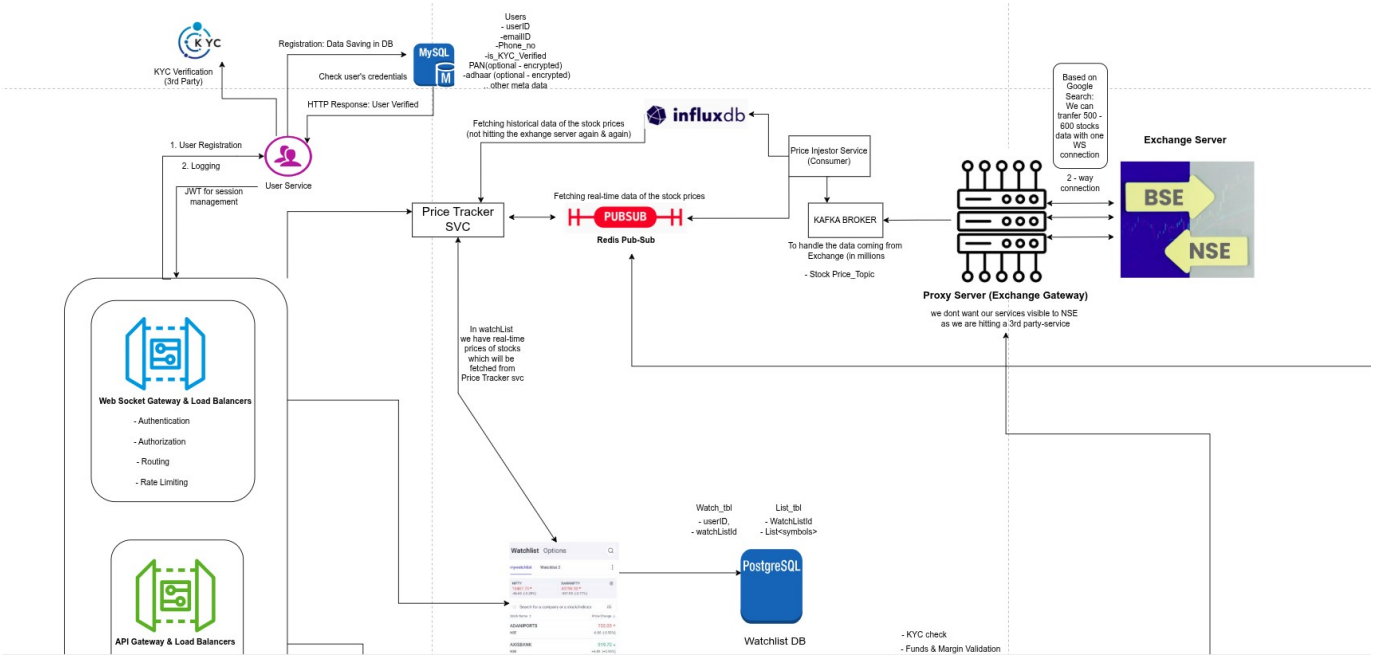
Scalability
- Target Users: Millions of users
- High number of concurrent orders during peak hours
- Distributed database for handling large-scale data

Availability
- System should prefer High Availability over strict consistency
- Eventual consistency model should be used

Latency
- Response time: 50–100 ms
- Real-time tracking updates with minimal delay

Security
- Secure authentication (JWT/OAuth)
- Encryption using HTTPS/TLS
- Secure payment gateways

Reliability
- Fault-tolerant system with redundancy
- Load balancing across servers



Outcome / Result -

After completing this experiment, we successfully designed and analyzed a scalable food delivery system with clearly defined functional and non-functional requirements. The system includes user management, restaurant browsing, order placement, real-time delivery tracking, and secure payment integration. The architecture is based on distributed system principles such as high availability, load balancing, and eventual consistency. The designed system is capable of handling large-scale traffic efficiently and can be used in real-world applications similar to modern food delivery platforms.